Experiment No -02 (Group A)

Title:-

Library Book Borrowing Record Management using Python

Problem statement:-

Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities:

• Compute the average number of books borrowed by all library members.

• Find the book with the highest and lowest number of borrowings in the library.

• Count the number of members who have not borrowed any books (denoted by a borrow count of 0).

• Display the most frequently borrowed book (i.e., the mode of borrow counts).After performing, determine the time and Space complexity of each operation

Pre-requisite:-

- Basic knowledge of **Python programming.**
- Understanding of **conditional statements** (if, elif, else).
- Knowledge of **loops** (for and while).
- Understanding of **functions** in Python.
- Knowledge of frequency counting (using **dictionaries** or **collections**).
- Understanding of average, maximum, minimum, and mode calculations.
- Basic understanding of **Time Complexity and Space Complexity** using Big-O notation.

Objective:-

Should be able to develop a Python program that manages library borrowing records by performing statistical analysis on borrow counts.

The program Should be able to calculates

- Average books borrowed by members
- Finding most/least borrowed books
- Counting members who borrowed zero books
- Finding the mode of book borrow counts

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. IDE :- Python 3

Outcome:-

Successfully calculated the average number of books borrowed. Identified the books with the maximum and minimum borrowings. Counted the number of members with zero borrowings. Determined the mode (most frequent borrow count).

Analyzed the time and space complexity of each operation, showing an overall time complexity of **O(n)** and space complexity of **O(n)**.

Theory:-

List:-

A List is a collection of ordered python objects separated by commas and changeable.

In Python List are written with square brackets.

To create a List in Python, place all the elements in a [ ] square brackets, separated by commas.

A List can have heterogeneous data items, a tuple can have string and list as data items as
well but list are commonly used as homogenous objects.

e.g. list1 = [“apple”,“banana”, „”herry”,“kiwi”,”orange”]

print(list1) Output:- [“apple”,“banana”, „”herry”,“kiwi”,”orange”]

print(list1[1]) Output :- banana

print(list1[-1]) Output :- orange

print(list1[1:3]) Output :- [„banana“,“cherry“]

print(list1[-3:-1]) Output :- [“cherry“,“kiwi“]

Dictionary :-

A dictionary is a collection which is unordered, changeable and indexed.

Creating a dictionary is as simple as placing items inside curly braces {} separated by comma.

Dictionary as a set of key: value pairs, with the requirement that the keys are unique.

Used to map each member to their borrow count, and each book to its total borrow count

e.g. d = { 1: "First", 2: "Second", "Third": 3 }

print(d)

Output:- { 1: "First", 2: "Second", "Third": 3 }

x = d[2]

print(x) Output:- Second

y = d.get(2)

print(y) Output:- Second

d["Third"] = 13

print(d)

Output:- { 1: "First", 2: "Second", "Third": 13 }

```
for z in d.values():
    print(z)
```

Output:-

First

Second

13

Counter:-

Helps find frequency (mode) of borrow counts for books.

Algorithm:-

- Step 1: Accept member names and number of books they borrowed using a dictionary
- Step 2: Accept book titles and number of times each was borrowed
- Step 3: Calculate average books borrowed: sum of books / number of members
- Step 4: Use max() and min() to find books with highest and lowest borrow count
- Step 5: Count members with 0 borrowed books using a loop
- Step 6: Use Counter to calculate frequencies and find mode of borrow counts

Time and Space Complexity:

Functionality	**Time Complexity**	**Space Complexity**	**Notes**
Compute average borrowed	O(n)	O(1)	n = number of members
Find max/min borrowed books	O(m)	O(k)	m = number of books, k = books with max/min borrow
Count members with 0 borrowed books	O(n)	O(1)	n = number of members
Most frequently borrowed book(s) (mode analysis)	O(m + k)	O(k + r)	k = distinct borrow counts, r = result books

Conclusion:-

This program helps the librarian to analyze borrowing statistics of library members and books. It efficiently uses Python dictionaries and the Counter class to perform the required computations, demonstrating basic data structure usage and time-space efficiency.

Viva Questions:-

1. What is a dictionary in Python?
2. What is a list in Python?
3. What is the difference between a list and a dictionary?
4. Which function is used to find the maximum borrow count?
5. Which function is used to find the minimum borrow count?
6. What is Time Complexity?
7. What is Space Complexity?